Unsupervised Learning

Lesson 8 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

27 November 2026 · reading time about 70 minutes

Contents

Lesson plan . . . 2
1. Why this lesson exists . . . 3
    1.1 Three problems . . . 3
2. k-means . . . 4
    2.1 One centre per group . . . 4
    2.2 Two moves, repeated . . . 5
    2.3 Six customers, by hand . . . 5
    2.4 Why it always stops, and where . . . 6
    2.5 How many groups: the elbow and the silhouette . . . 7
    2.6 Checking against the truth: the adjusted Rand index . . . 9
3. When groups are not round . . . 10
    3.1 The sessions . . . 10
    3.2 Why k-means must fail here . . . 11
    3.3 Hierarchical clustering: a tree of merges . . . 11
    3.4 DBSCAN: crowded places, not centres . . . 12
    3.5 Choosing `eps` . . . 14
    3.6 DBSCAN on the sessions . . . 14
    3.7 What the silhouette cannot see . . . 15
    3.8 Choosing among the three . . . 15
4. Principal component analysis . . . 16
    4.1 Eight columns, not eight independent numbers . . . 16
    4.2 The widest shadow, on two columns . . . 17
    4.3 Any number of columns: the eigenvectors of the covariance matrix . . . 19
    4.4 How many components . . . 19
5. Anomalies that break the pattern . . . 20
    5.1 Every value ordinary . . . 20
    5.2 Reconstruction error . . . 21
    5.3 37 of 40 . . . 23
    5.4 Why a 2-D picture misses them . . . 23
6. Choosing among today's methods . . . 24
Summary . . . 25
    Homework . . . 25
    Notation used in this lesson . . . 25

Further reading 26

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 7 discussed; what changes when there is no y; three problems	Slides 2–6
0:10–0:40	30	k-means: one centre per group, two moves, six customers by hand, a bad start, how many groups, the check against the truth	Slides 7–21
0:40–1:05	25	**Notebook 01** — k-means by hand	Slide 22
1:05–1:20	15	When groups are not round: the sessions, why k-means must fail, a tree of merges	Slides 23–27
1:20–1:32	12	**Break**	Slide 28
1:32–1:50	18	DBSCAN: core, border and noise; choosing `eps`; what the silhouette cannot see	Slides 29–36
1:50–2:10	20	**Notebook 02** — merge trees and DBSCAN	Slide 37
2:10–2:35	25	Principal component analysis (PCA): the widest shadow; fraud that breaks the pattern; reconstruction error; why a 2-D plot misses it	Slides 38–51
2:35–2:55	20	**Notebook 03** — PCA and anomalies	Slide 52
2:55–3:00	5	What to take away; homework	Slides 53–54
	180	**Total**	**54 slides, 3 notebooks**

1. Why this lesson exists

Every method in this course so far learned from pairs: an input x and a target y, such as a price, a class, a failure or not. **Unsupervised learning** removes y. What is left is a table of measurements with no answer key, and the task changes from "predict the label" to "describe the structure": which records look alike, which few numbers summarise the many, which records do not belong.

This matters in practice as much as in principle. Most data an organisation collects was never labelled by anyone, and labelling it is slow, expensive, or not even well defined: "which of these customers are alike?" has no answer until somebody groups them and looks. Unsupervised methods are often the *first* thing run on a new dataset, and supervised learning the second, once the first look has suggested what the targets or the useful features are.

This version of the lesson keeps the mathematics light: every method is built from a picture and a worked example, not from a proof. The proofs, and a few more methods, are in the advanced version of this lesson, in `Advanced/08_unsupervised_learning_advanced/`, for anyone curious; nothing there is examined.

1.1 Three problems

The lesson follows **Aurora**, a fictional online retailer, through three questions, one per notebook. None of them has a y.

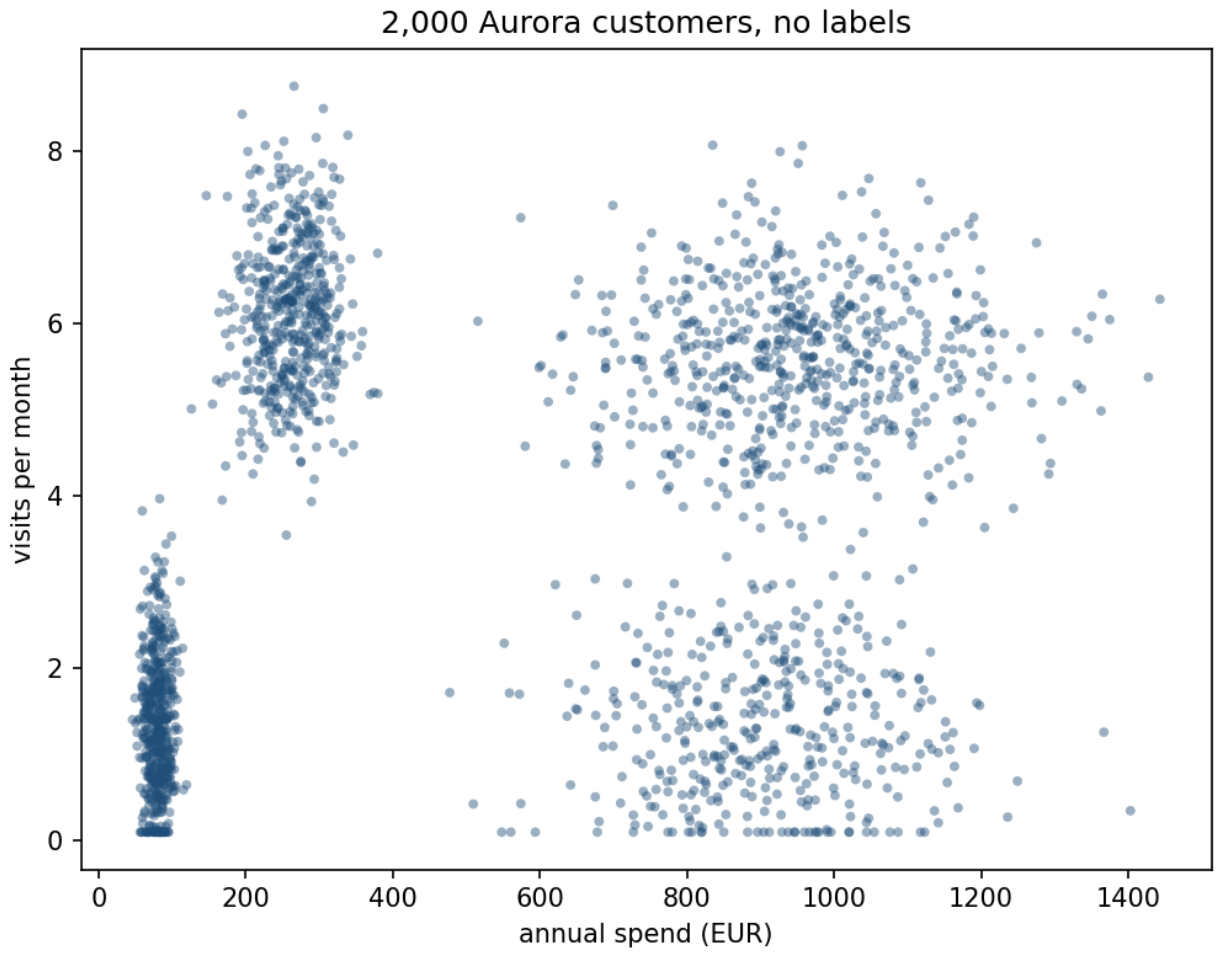
1. **Group 2,000 customers** for a marketing campaign, from two numbers per customer: annual spend and visits per month (**k-means**, notebook 01).
2. **Tell bots from humans** in 1,500 website sessions, where the two kinds have the same *average* behaviour and differ only in how much they vary (**hierarchical clustering and DBSCAN**, notebook 02).
3. **Summarise an 8-column account table, and catch 40 fraudulent accounts hidden in it**, accounts that look ordinary on every single column (**principal component analysis and reconstruction error**, notebook 03).

Every dataset is synthetic, and the structure that generated it is published in `Notebooks/retail_data.py` as `TRUE_*` constants. That lets this lesson check each method against a truth that Aurora's own analysts would never have. Hold on to that asymmetry: it is what makes the lesson teachable, and it is exactly what will be missing the first time any of these methods runs on real data.

The number to carry out of the lesson: **37 of the 40 disguised accounts are caught by reconstruction error, and a 2-D scatter plot of the same accounts would point at the wrong ones.** Section 5 explains both halves.

---

2. k-means



2,000 Aurora customers, no labels. Look for the four clouds: they are visible before any algorithm runs, and they are exactly the kind of structure k-means is built for.

2.1 One centre per group

The idea first, with no symbols. If customers really fall into a few groups, each group can be described by one representative point, its "typical customer", which we call the group's **centre**. A good grouping is one in which every customer is close to the centre of their own group. k-means turns that sentence into an algorithm: it looks for k centres, and gives every customer to the nearest one.

Two choices are hidden in that sentence, and both matter.

- **k, the number of groups, is chosen by you.** The algorithm does not discover it. Section 2.5 is about how to choose it.
- **"Nearest" means straight-line distance.** So the columns must be on comparable scales. Annual spend runs into hundreds of euros and visits per month into single figures; measured raw, a difference of 50 euros would outweigh a difference of 5 visits a month, and the grouping would be decided by spend alone. Notebook 01 standardises both columns first (lesson 2), so

that each counts in units of its own spread.

2.2 Two moves, repeated

The algorithm alternates two moves until nothing changes.

1. **Assign.** Give every point to its nearest centre.
2. **Move.** Put every centre at the mean of the points now assigned to it. The mean is what gives the method its name.

To start, the centres need somewhere to be: the simplest choice is k points picked at random from the data.

How good a grouping is gets measured by one number, the **within-cluster sum of squares** (WCSS): for every point, the squared distance to the centre of its own group, added up over all points. Draw a line from every point to its centre; WCSS is the sum of the squared lengths of those lines. In symbols, if $c(i)$ is the group of point i and μ_j the centre of group j,

$$J = \sum_{i=1}^{m} \|x_i - \mu_{c(i)}\|^2$$

scikit-learn reports the same number as `inertia_`. The letter J is the course's symbol for the quantity a method makes small (lessons 3 and 4); here there is no y in it, only the data and the centres.

2.3 Six customers, by hand

Six customers, with spend in hundreds of euros a year and visits per month, so that the two numbers are on comparable scales without rescaling:

Customer	Spend (hundreds of EUR)	Visits per month
A	1	1
B	1	2
C	2	1
D	8	6
E	9	6
F	8	7

Start the two centres on A and B, a deliberately poor choice, since both are small occasional buyers.

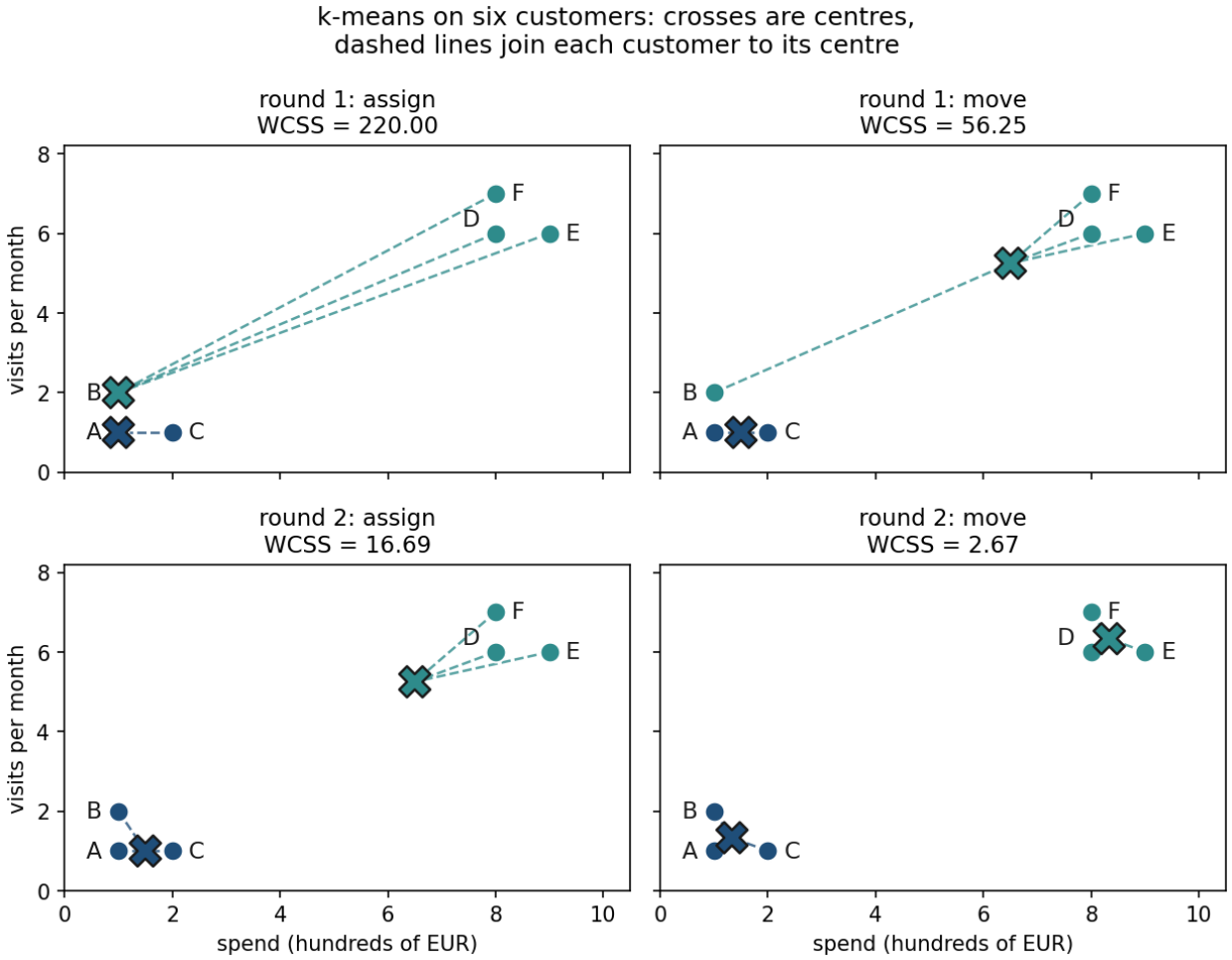
Round 1, assign. Each customer goes to the nearer of A $(1, 1)$ and B $(1, 2)$, comparing squared distances. C is nearer A: $1^2 + 0^2 = 1$ against $1^2 + 1^2 = 2$. D, E and F are all nearer B, which is one visit higher; D, for instance, is at $7^2 + 4^2 = 65$ from B and $7^2 + 5^2 = 74$ from A. So the groups are AC and BDEF. The squared lengths of the six lines are 0 (A), 1 (C), 0 (B), 65 (D), $8^2 + 4^2 = 80$ (E) and $7^2 + 5^2 = 74$ (F): **WCSS = 220**.

Round 1, move. The AC centre goes to the mean of A and C, $(1.5, 1)$. The BDEF centre goes to the mean of four points, $\left(\frac{1+8+9+8}{4}, \frac{2+6+6+7}{4}\right) = (6.5, 5.25)$. The lines shorten: A and C are now 0.25 each, D $1.5^2 + 0.75^2 = 2.8125$, E $2.5^2 + 0.75^2 = 6.8125$, F $1.5^2 + 1.75^2 = 5.3125$, and B, left far behind, $5.5^2 + 3.25^2 = 40.8125$: **WCSS = 56.25**.

Round 2, assign. B is now much nearer the first centre: $0.5^2 + 1^2 = 1.25$ against 40.8125. It switches, and the groups become ABC and DEF. Its line shrinks from 40.8125 to 1.25: **WCSS = 16.6875**.

Round 2, move. The centres go to $(\frac{4}{3}, \frac{4}{3})$ and $(\frac{25}{3}, \frac{19}{3})$. Each triangle of three customers now contributes $\frac{2}{9} + \frac{5}{9} + \frac{5}{9} = \frac{4}{3}$: **WCSS = 8/3 = 2.6667**.

Round 3 assigns everybody to the same group as before, so the centres do not move, and the algorithm stops at ABC and DEF.



The four stages of the worked example. Crosses are the two centres, dashed lines join each customer to its centre, and WCSS, in each title, is the sum of the squared lengths of the dashed lines. Look at B: in the second panel it is still tied to a centre far away, and in the third it switches.

2.4 Why it always stops, and where

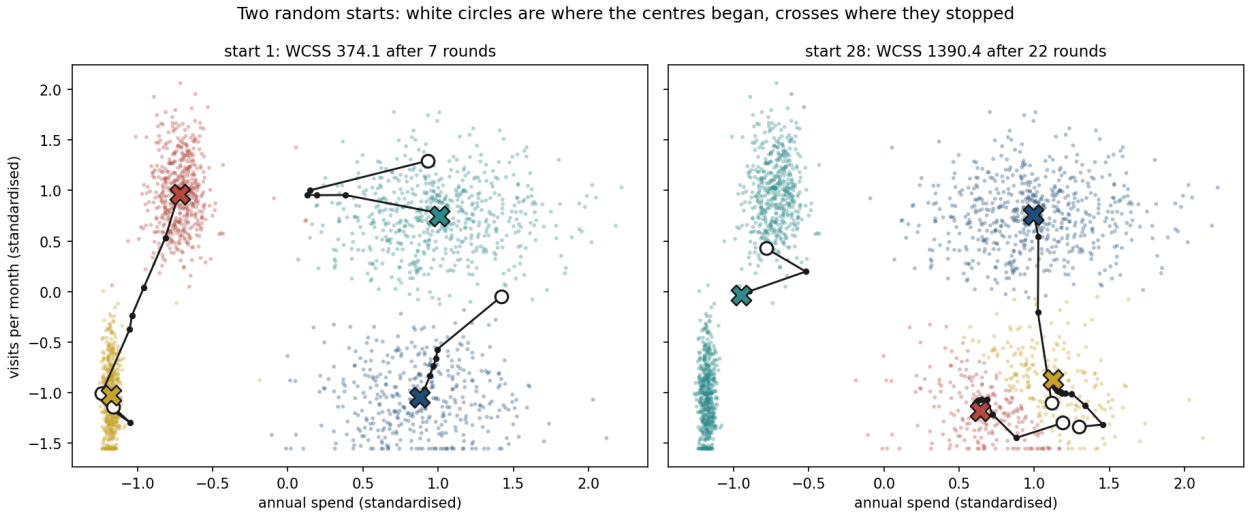
Read the WCSS sequence again: 220, 56.25, 16.6875, 2.6667. It never went up, and that is not luck.

- **An assign step cannot raise the WCSS.** Each point moves to its nearest centre, so its line can only get shorter or stay the same.
- **A move step cannot raise it either.** For a fixed group of points, the mean is the single point that makes the sum of squared distances to them smallest. Take one coordinate: $\sum_i (x_i - \mu)^2$

is a parabola in μ, and its derivative $-2\sum_i (x_i - \mu)$ is zero exactly when μ is the mean. So moving the centre to the mean can only shorten the lines, or leave them as they are.

The score can only fall or stay put, and there are only finitely many ways to split m points into k groups, so the algorithm must reach a grouping that repeats, and there it stops. The handout of the advanced lesson writes this argument out in full.

What the argument does **not** say is that the stopping point is the best grouping. Notebook 01 runs the same code on the 2,000 standardised customers from 30 different random starts. 29 of them stop at a WCSS of **374.1**; one stops at **1,390.4**, more than three times worse, and stays there, because from where it stands no single assign or move lowers the score.



Two random starts on the same data. White circles are where the four centres began, black paths are where they travelled, crosses where they stopped. Look at the right panel: two centres share the group at the bottom right while one centre covers both groups on the left. There, assigning and moving change nothing any more, so the algorithm stops, although a better grouping exists.

The remedy is cheap: run k-means several times from different starts and keep the run with the lowest WCSS. scikit-learn's `KMeans` does this by default with `n_init=10` restarts, and it also chooses its starting centres more carefully than at random, with a method called **k-means++** that prefers starting points far from those already chosen. Notebook 01 checks that the ten-line version and scikit-learn reach the same WCSS, 374.0919, to four decimal places.

The predictable mistake here is to read "the algorithm converged" as "the algorithm found the answer". The instinct is reasonable: in lesson 3, the least-squares cost was a bowl with exactly one bottom, so stopping meant arriving. k-means' score has many local minima, and stopping only means that no single step helps any more.

2.5 How many groups: the elbow and the silhouette

k is an input. Two diagnostics help choose it, and they ask different questions.

The elbow. WCSS always falls as k grows: with one group per point, every line has length zero and WCSS is exactly 0. So the value alone says nothing, and choosing the k with the lowest WCSS would always choose the largest k tried. What helps is *where the curve stops falling steeply.* Past

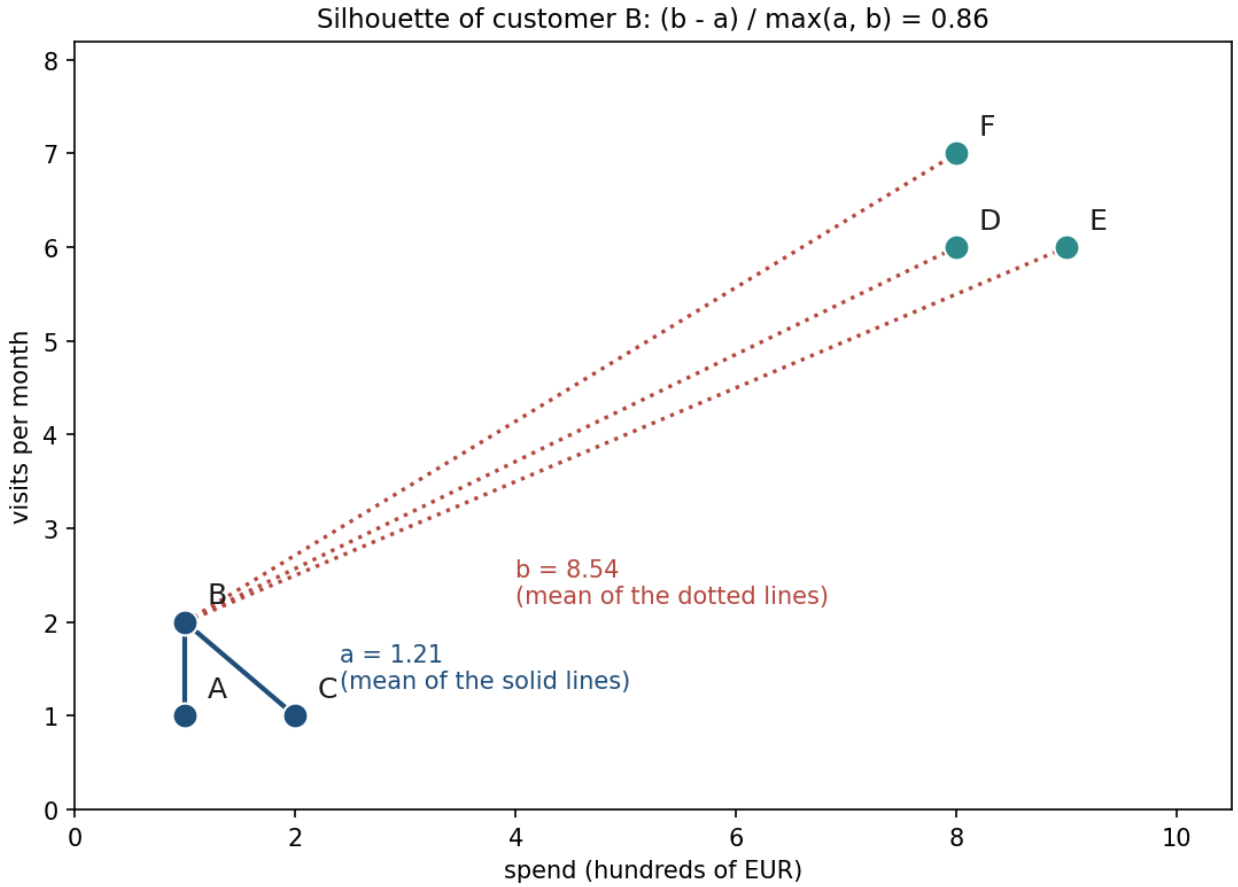
the right number of groups, an extra centre only splits a group that was already fine, and buys little.

The silhouette. For one point, let a be its average distance to the other points of its own group, and b its average distance to the points of the nearest *other* group. Its silhouette is

$$s = \frac{b - a}{\max(a, b)}$$

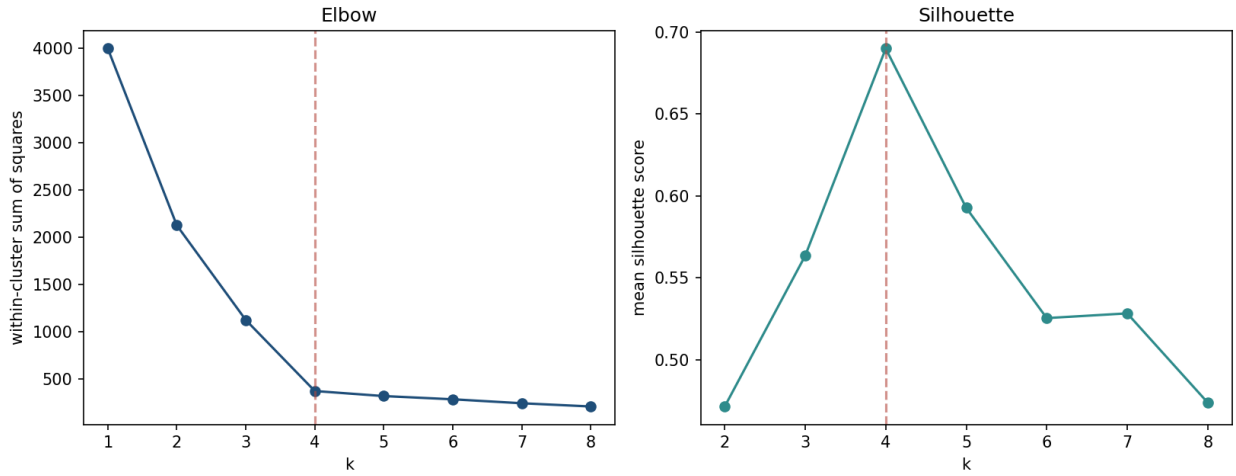
which lies between -1 and 1. It is near 1 when the point sits well inside its own group, far from any other (a much smaller than b); near 0 when it sits on a border ($a \approx b$); negative when it is closer, on average, to another group than to its own. The silhouette of a whole grouping is the average over all points, and the k with the highest average is the one to prefer.

Worked example. Customer B in the final grouping ABC, DEF. Its distances to the other members of its own group are 1 (to A) and $\sqrt{2} = 1.414$ (to C), so $a = 1.207$. Its distances to D, E and F are $\sqrt{65} = 8.062$, $\sqrt{80} = 8.944$ and $\sqrt{74} = 8.602$, so $b = 8.536$. Then $s = (8.536 - 1.207)/8.536 = 0.859$: B is well inside its group.



The silhouette of one customer, drawn. Look at the two kinds of line: a is the mean length of the solid blue lines to B's own group, b the mean length of the dotted red lines to the other group. When the solid lines are much shorter than the dotted ones, the silhouette is close to 1.

On the 2,000 customers, notebook 01 computes both diagnostics for k from 1 to 8.

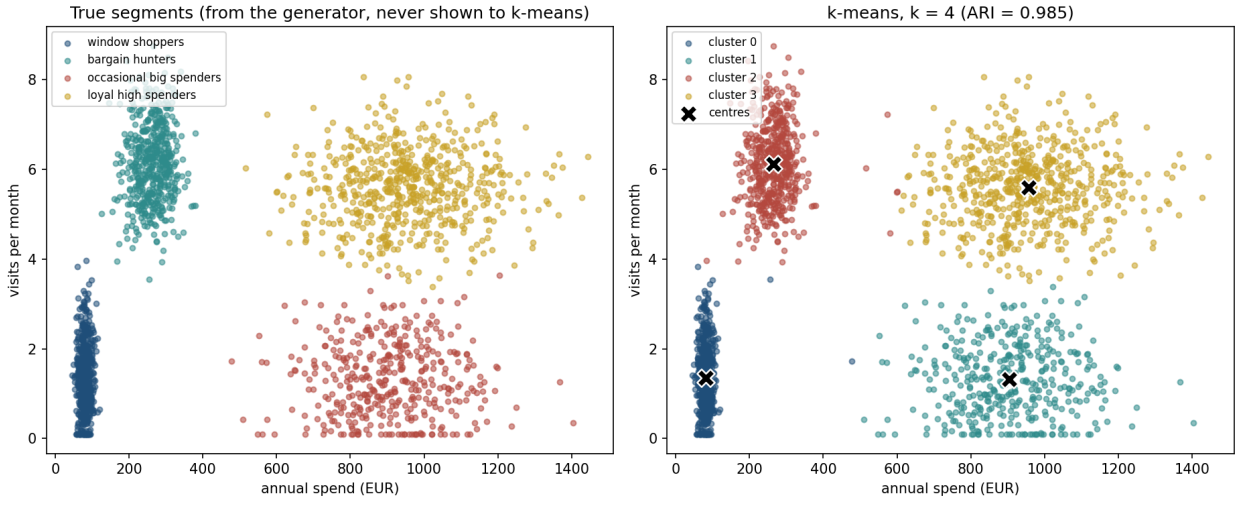


Left: WCSS against k. Look for where it stops falling steeply: after $k = 4$, the dashed line. Right: the mean silhouette, highest at the same $k = 4$, with 0.690. Reading a bend takes judgement; reading a maximum does not, which is why the silhouette is easier to act on.

Both point to 4, the number of segments the data was built with. They agree because the data really is four round, well separated clouds; section 3.7 shows a case where the silhouette alone points the wrong way.

2.6 Checking against the truth: the adjusted Rand index

This lesson knows which segment generated each customer, so one more check is possible. The **adjusted Rand index** (ARI) compares two groupings of the same points without caring what the groups are called. Take every pair of points and ask whether the two groupings agree about the pair: together in both, or apart in both. The share of pairs on which they agree is the **Rand index**. Even a random grouping agrees on many pairs, since most pairs are apart in both, so the *adjusted* version rescales it: a random grouping scores about 0, an identical one scores 1, and a grouping worse than random can score below 0.



Left: the true segments, from the generator, never shown to k-means. Right: k-means with $k = 4$,

centres marked with crosses. Look for the handful of customers coloured differently from their neighbours on the left: they are the disagreements, where two segments touch.

k-means scores an ARI of **0.985**: almost exactly the true segments. Note what kind of number this is. The silhouette is an **internal** measure: it uses only the data and the grouping, so Aurora could compute it. The ARI is an **external** measure: it needs a true grouping from outside, which Aurora does not have and never will. Every ARI in this lesson exists only because the data is synthetic.

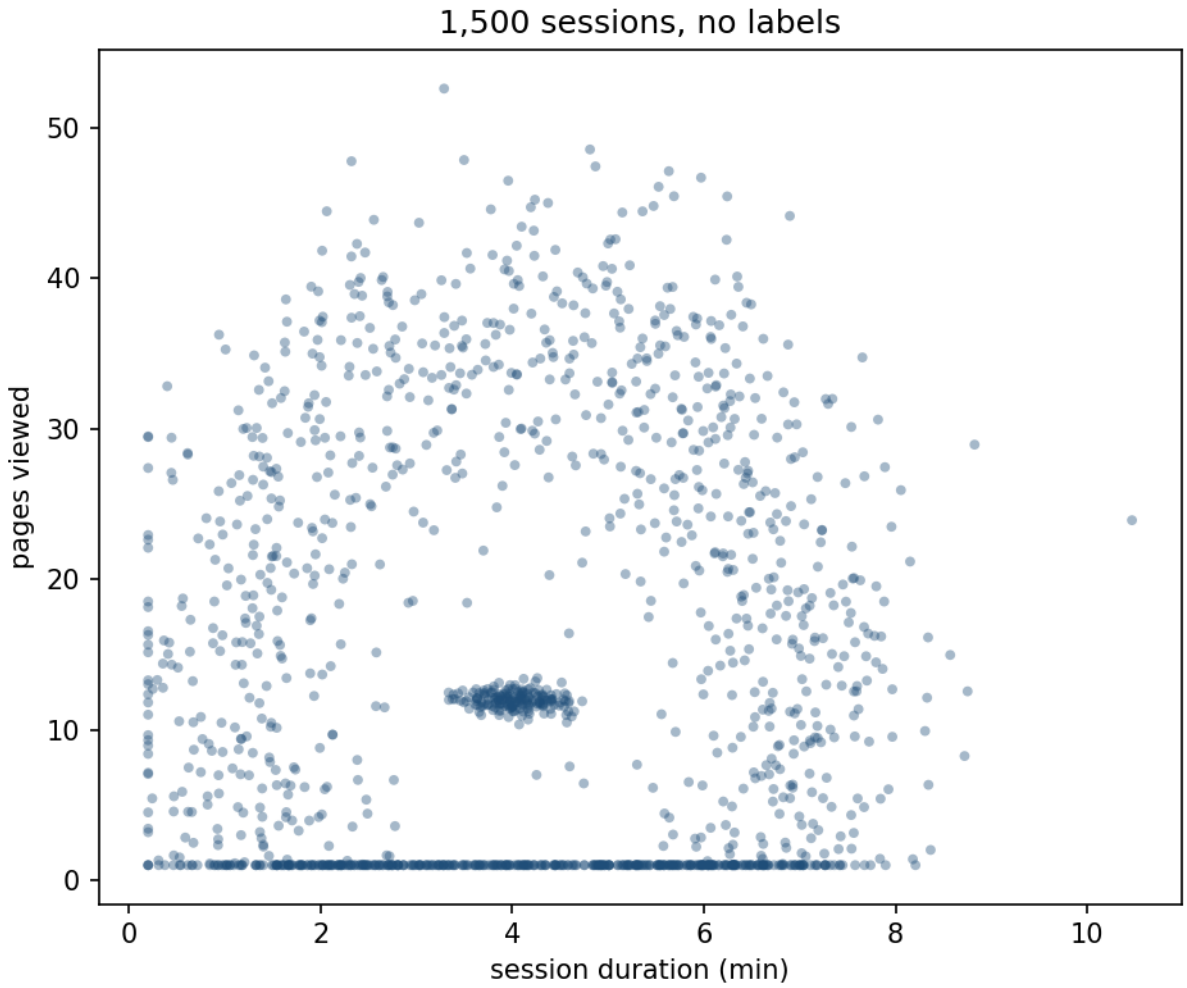
The predictable mistake after this section is to conclude that k-means is a reliable way to find groups. The reasoning is sound as far as it goes: on a few round clouds of similar spread, k-means is fast and close to the best possible. What fails is the assumption under it, and section 3 is about that.

---

3. When groups are not round

3.1 The sessions

Aurora's security team suspects that some website sessions are run by scripts (bots). For each of 1,500 sessions they log its duration and the number of pages viewed.



1,500 sessions, no labels. Look at the middle: a dense smudge sits inside a much larger, looser ring, and both are centred on the same place.

A script does the same thing every time, so the bots pile up in a tight spot. People vary, so they spread out around it. `retail_data.py` makes 12% of the sessions bots (180 of 1,500) and gives both groups the same average duration and page count; only the spread differs. The natural split is *close to the middle* against *far from it*.

3.2 Why k-means must fail here

k-means with $k = 2$ scores an ARI of **−0.046**: no better than chance. All 180 bots land in one cluster, but so do 780 humans.

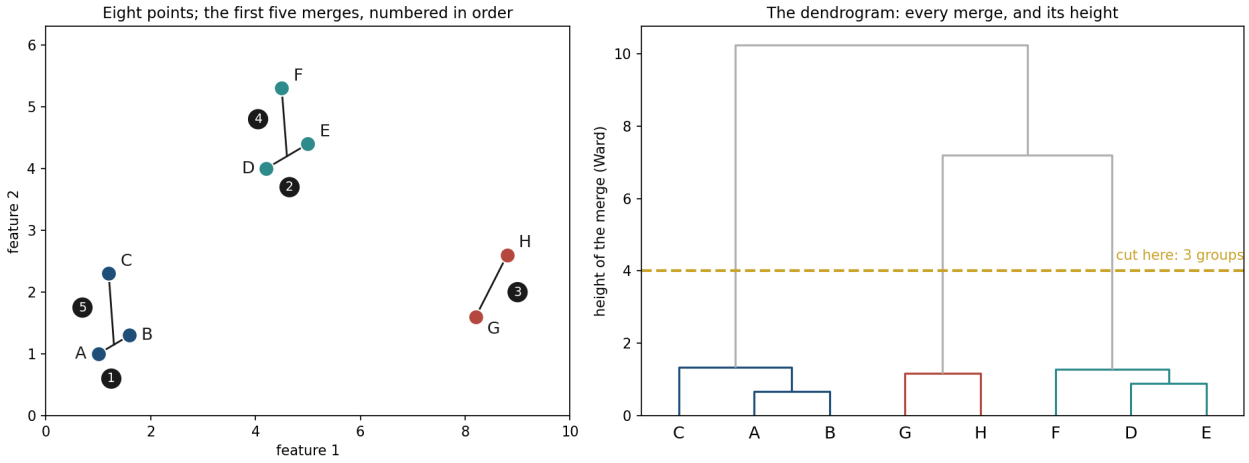
This is not bad luck with the starting point. With two centres, every point goes to the nearer one, and the points equally far from both form a straight line halfway between them. So the two groups k-means can produce are always the two sides of a straight line. No straight line separates a core from the ring around it, so the best k-means can do is cut the picture in half, which cuts the ring in half too.

3.3 Hierarchical clustering: a tree of merges

A different idea. Start with every point as a group of its own. Merge the two groups that are closest. Repeat until everything is one group. Every merge is recorded, and the record is a tree called a **dendrogram**: the height at which two branches join says how far apart the two groups were when they merged. Cutting the tree with a horizontal line gives a grouping, and the number of branches the line crosses is the number of groups. So choosing where to cut is choosing k, and one tree serves every k.

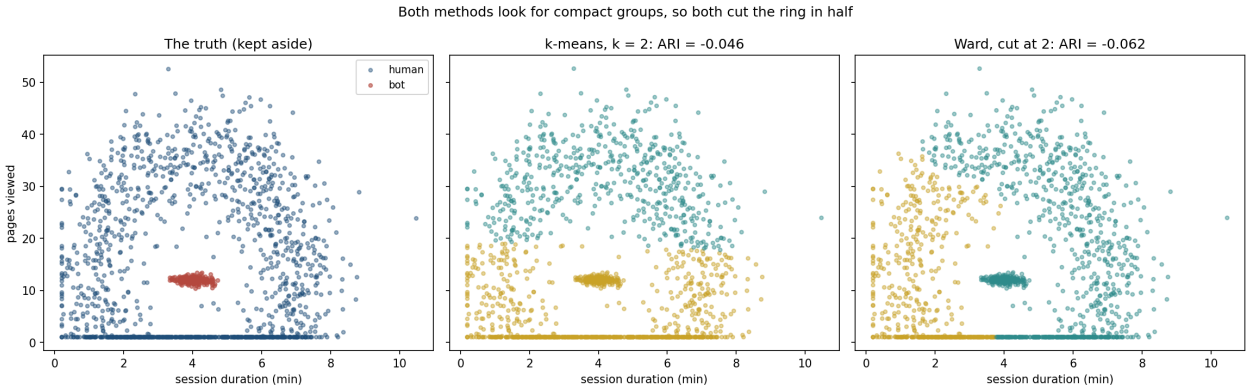
"Closest" needs a rule once groups have more than one point; that rule is called the **linkage**. scikit-learn's default, **Ward linkage**, merges the two groups whose union adds the least to the within-cluster sum of squares of section 2: the merge that keeps the groups most compact. For two single points, that is simply the two nearest, and the height of their merge is the distance between them.

Worked example. Eight points in three loose clumps, ABC, DEF and GH (notebook 02 lists their coordinates). The first three merges join the closest pairs: A with B at height 0.671, which is their distance, $\sqrt{0.6^2 + 0.3^2} = \sqrt{0.45}$; then D with E at 0.894; then G with H at 1.166. Then F joins DE at 1.275, and C joins AB at 1.333. That leaves three groups, and the next merge must bridge a real gap: its height jumps to 7.193.



Left: the eight points, with the first five merges drawn as numbered segments. Right: the dendrogram of the same merges. Look at the long vertical stretch between heights 1.3 and 7.2 where nothing merges: that gap is the tree's way of saying "three groups", and the dashed cut at height 4 crosses exactly three branches.

On the sessions, cut at two groups, hierarchical clustering with Ward linkage scores an ARI of **−0.062**, no better than k-means, and for the same reason: Ward merges whatever keeps groups compact, which is what k-means rewards too, so it ends up with two compact halves. A different procedure, the same idea of what a group is.



The truth, k-means and Ward side by side. Look at where each method puts the bots: k-means puts them with the bottom of the ring, Ward with its right-hand side. Both draw two compact halves and neither separates core from ring.

Other linkages exist: merging by the closest pair of points, or by the farthest, or by the average distance. On this data none of them separates the bots either; the advanced lesson compares all four.

3.4 DBSCAN: crowded places, not centres

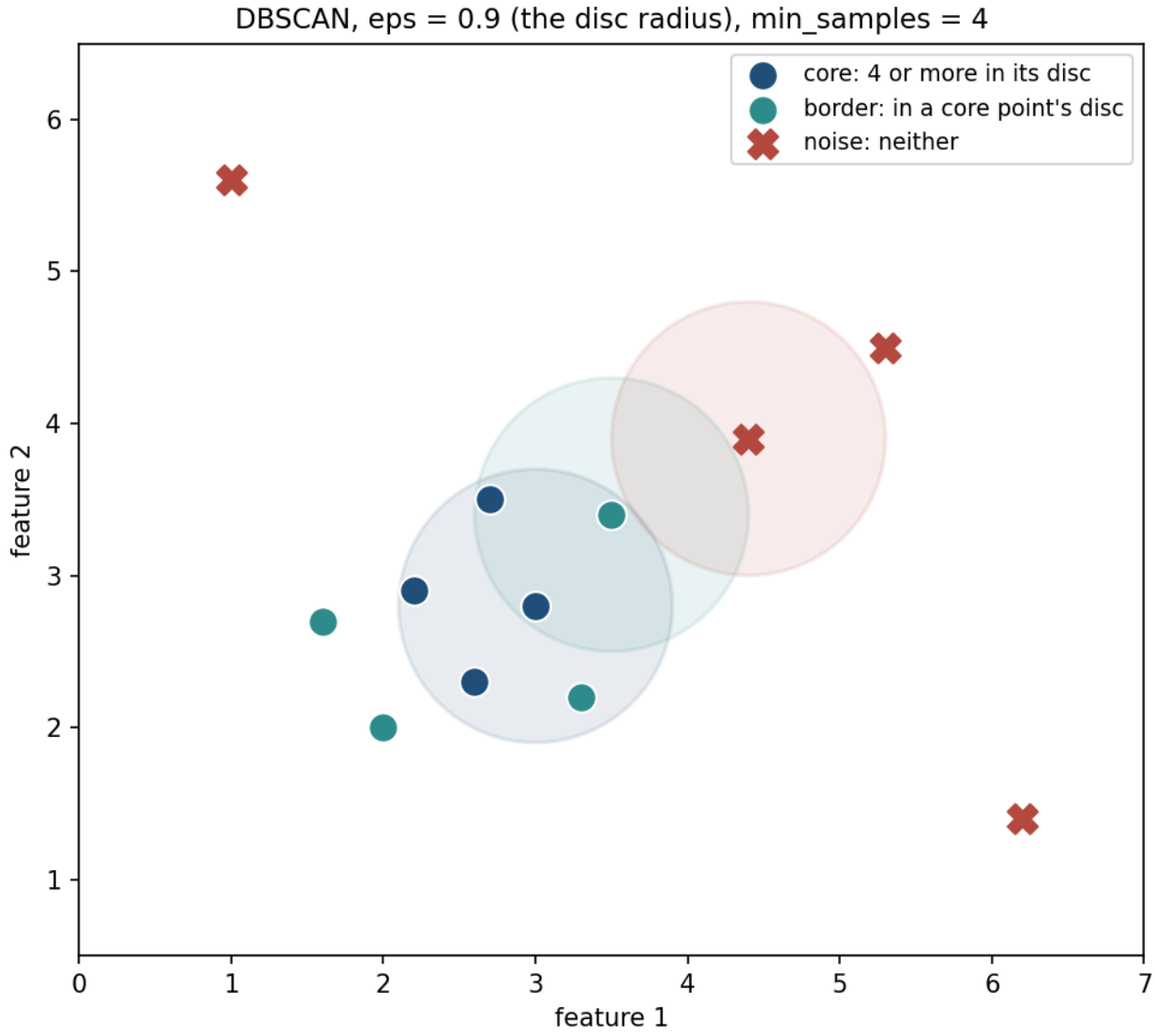
DBSCAN (density-based spatial clustering of applications with noise) asks a different question about each point: *is it in a crowded place?* It looks at the disc of radius `eps` around each point.

- A point is a **core point** if at least `min_samples` points lie in its disc, **the point itself included** (scikit-learn counts it).

- A point that is not core, but lies in the disc of a core point, is a **border point**: it joins that core point's group, but cannot extend the group further.
- Every other point is **noise**, and belongs to no group. scikit-learn labels it -1.

A group is a chain of core points, each inside another's disc, together with the border points around them. Nothing here mentions a centre or a shape, so a group can be a ring, a band, or any connected crowded region. And DBSCAN does not need k: the number of groups is however many chains there turn out to be.

Worked example. Twelve points, `eps = 0.9`, `min_samples = 4`. Four points have four to six points in their discs, counting themselves: they are core. Four more have three, too few to be core, but each lies within 0.9 of a core point: they are border. The last four have nothing but themselves within 0.9: noise. One group, plus four noise points.

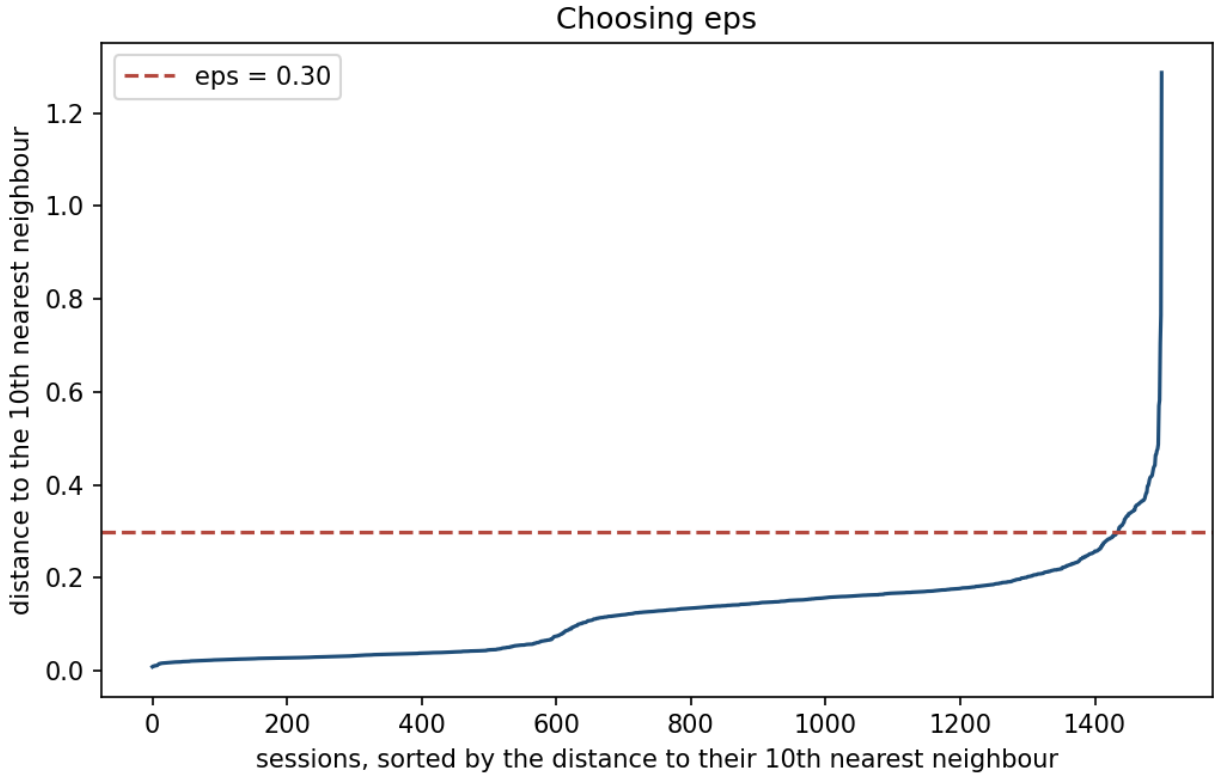


Core, border and noise, drawn. The three shaded discs are one of each kind. Look at the rust disc: its noise point is only a little farther than 0.9 from the group, but no core point is within its reach, so it joins nobody.

3.5 Choosing `eps`

`eps` is the parameter that matters most. Too small, and even the dense bot core breaks up into noise. Too large, and the discs of the ring reach into the core and chain everything into one group.

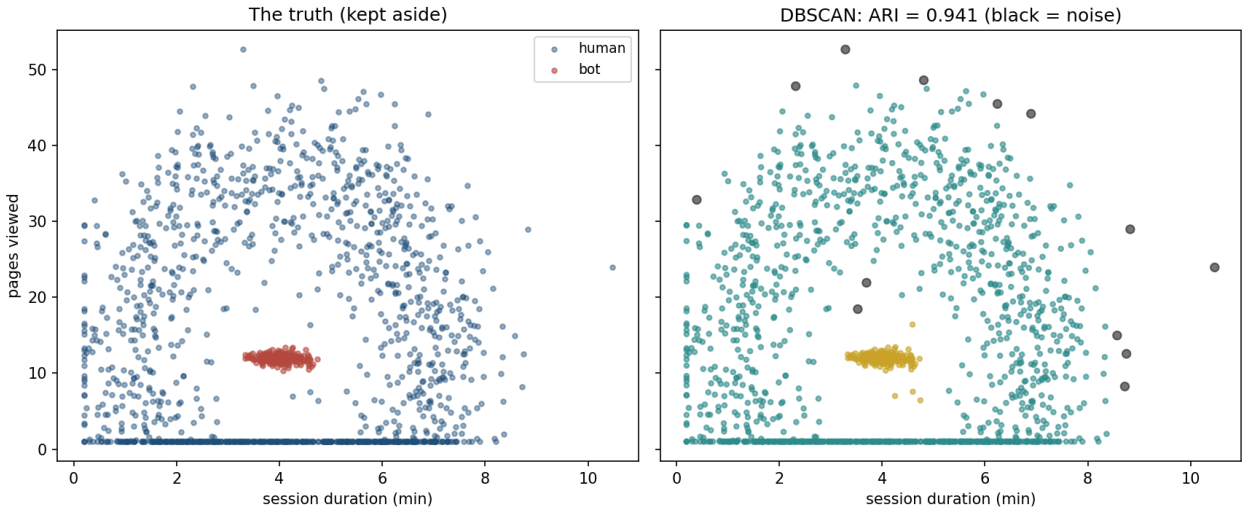
A **k-distance plot** suggests a value. For every point, take the distance to its 10th nearest neighbour (10 being the `min_samples` used), and sort those distances. Points in crowded places have short distances, points in sparse places long ones, and the sorted curve bends upward where crowded gives way to sparse. Put `eps` near the bend.



The 1,500 sessions sorted by the distance to their 10th nearest neighbour, in standardised units. Look for the bend at the right-hand end: the dashed line at 0.30 sits just above the flat part, and only 65 sessions lie above it.

3.6 DBSCAN on the sessions

With `eps = 0.30` and `min_samples = 10`, DBSCAN scores an ARI of **0.941**. All 180 bots form one group; 1,303 of the 1,320 humans form the other; 4 humans are placed with the bots; 13 sessions are noise.



The truth, and DBSCAN's answer. Look at the black points on the right: they are the 13 sessions DBSCAN calls noise, scattered around the thin outer edge of the ring and in the gap around the core.

All 13 noise sessions are humans, and they sit where the data is thin: two in the gap between the bot core and the ring, eleven on the ring's sparse outer edge. DBSCAN's answer for them is not "bot" or "human" but *not confidently either*, which is more useful to a security analyst than a forced label.

3.7 What the silhouette cannot see

Aurora would not have the true groups, so it could not compute an ARI. It could compute the silhouette, which needs only the data and the grouping:

Grouping	Silhouette (internal)	ARI (external)
k-means, $k = 2$	0.401	-0.046
DBSCAN	-0.106	0.941

The silhouette prefers the wrong answer. Two compact halves are exactly what it rewards, so k-means' useless split scores well. DBSCAN's correct answer scores below zero, because a ring is not compact: 99.2% of the ring's points have a negative silhouette, since on average they are closer to the bot core in the middle than to the rest of their own ring, most of which lies across the circle from them.

The predictable mistake: with no labels, the silhouette is the only score available, so it is natural to choose whichever grouping scores highest. The silhouette, like WCSS, measures compactness; it cannot tell you that compactness is the wrong question for the data in front of you. Only looking at the data, or knowing what the groups should mean, can.

3.8 Choosing among the three

Method	A group is	Needs k?	Fails when
k-means	the points nearest one centre	yes	groups are not round, or very different in size or spread
Hierarchical, Ward	a branch of the merge tree	chosen by the cut	as k-means: Ward rewards compactness too
DBSCAN	a crowded connected region	no	groups differ a lot in density: one `eps` cannot suit both

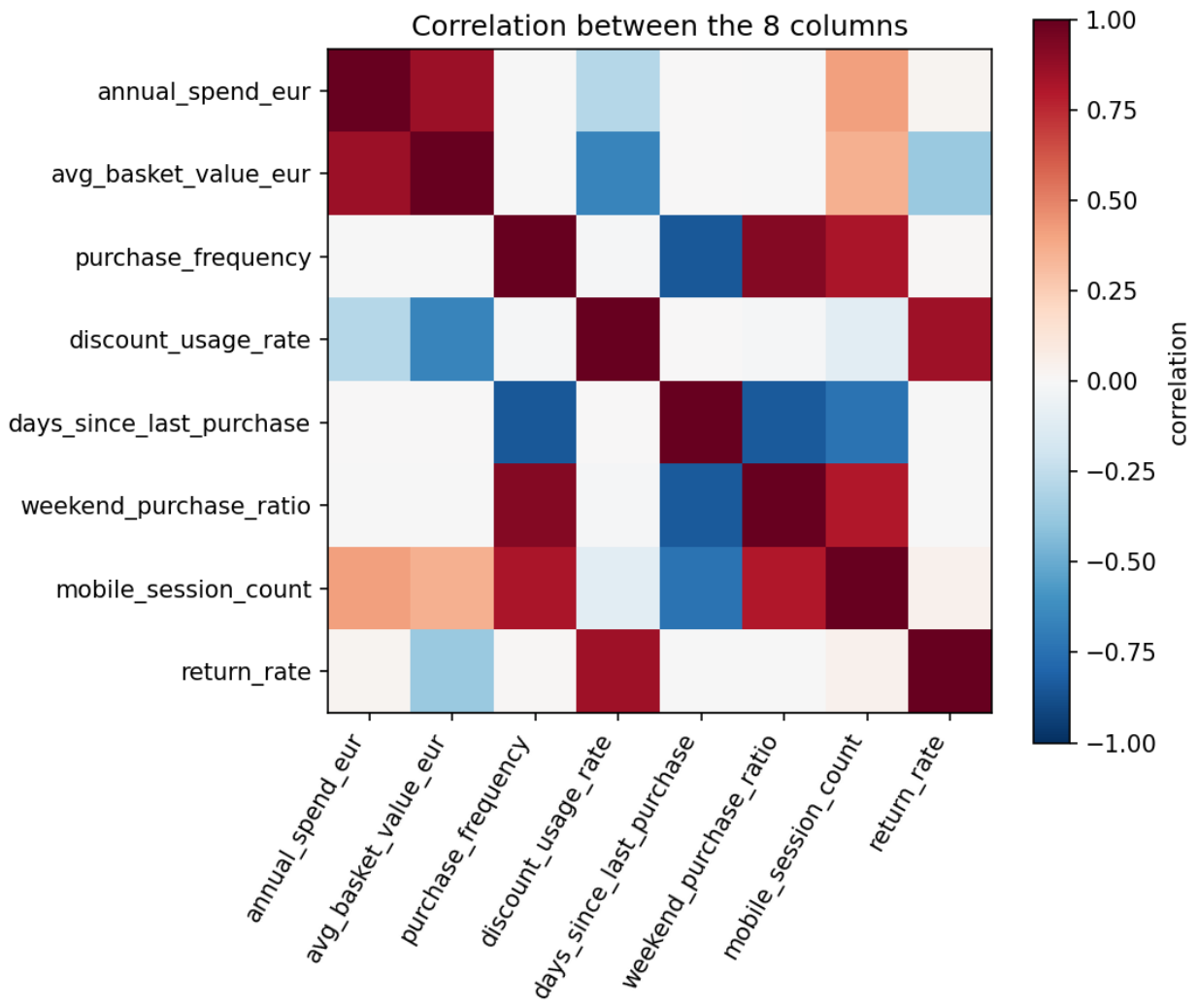
Hierarchical clustering earns its place when the tree itself is informative: it shows the groups at every scale at once, and on a few hundred points it can be read by eye.

---

4. Principal component analysis

4.1 Eight columns, not eight independent numbers

Aurora's account table has 8 columns: annual spend, average basket value, purchase frequency, discount usage, days since the last purchase, weekend share, mobile sessions, return rate. If the eight were unrelated, none could be dropped without losing information. They are not unrelated.



Correlations between the 8 standardised columns: dark red is strongly positive, dark blue strongly negative. Look at annual spend and average basket value: 0.859. Several columns move together, which means fewer than eight numbers are really varying.

`retail_data.py` knows why: three hidden tendencies (how much a customer likes to spend, how engaged they are, how sensitive to price) drive all eight columns. **Principal component analysis** (PCA) finds, from the data alone, a few new columns that carry almost all of the variation.

4.2 The widest shadow, on two columns

Take two of the columns, annual spend and average basket value, standardised (mean 0, standard deviation 1), with correlation $r = 0.859$. The points lie along a tilted band.

Pick a direction through the middle of the cloud and drop every point onto it at a right angle, like a shadow cast on a line. The position of each shadow along the line is one number per account: a one-number summary of two columns. Some directions spread the shadows widely, others squash them together. A summary that keeps the accounts apart keeps the information that tells them apart, so the best one-number summary is the direction where the shadows **spread the most**,

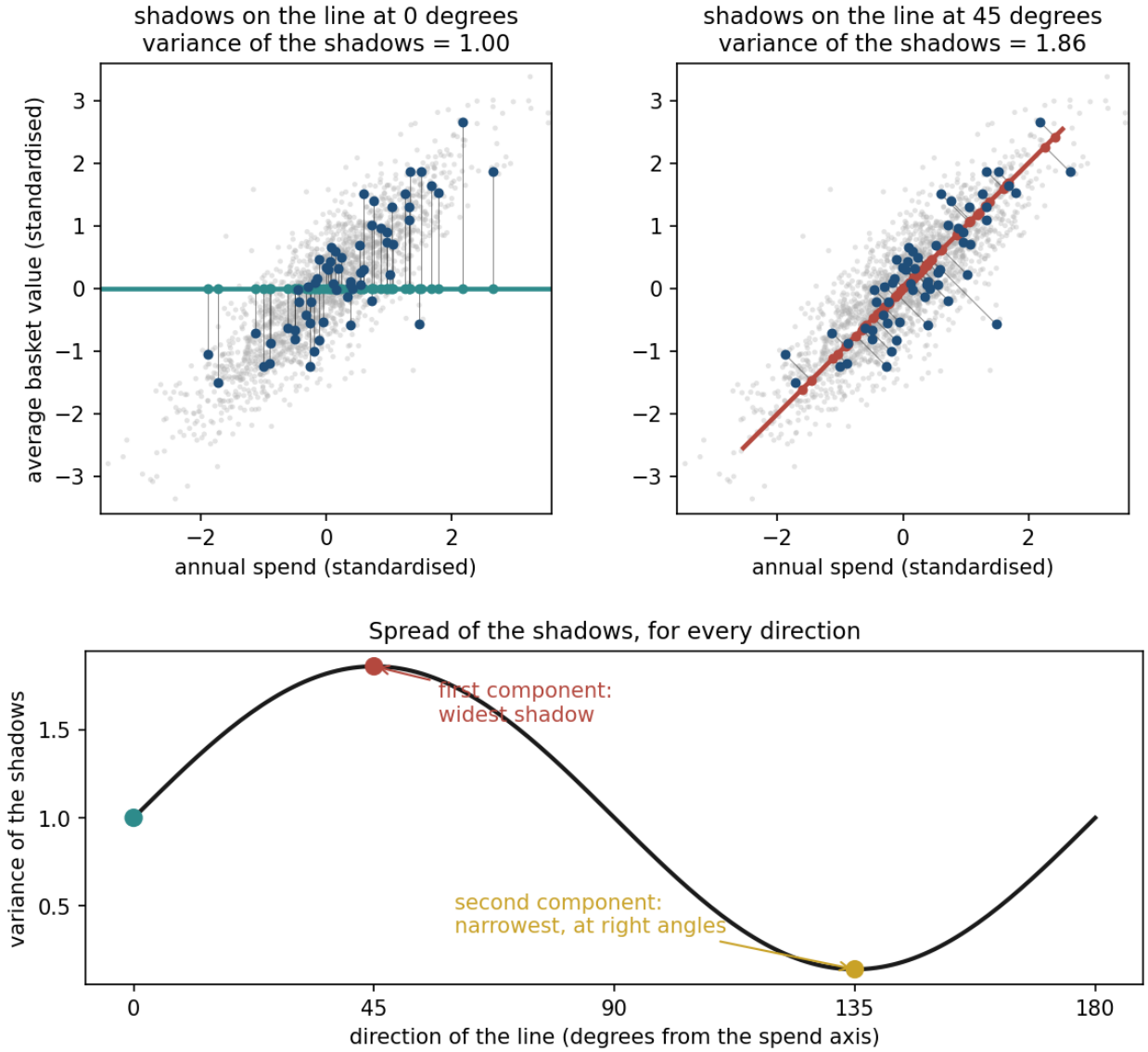
measured by their variance. That direction is the **first principal component**; the **second** is the direction at right angles to it.

Here the best direction can be found with two lines of algebra. Write the direction at angle θ from the spend axis as $u = (\cos\theta, \sin\theta)$. The shadow of a point $x = (x_1, x_2)$ is $z = x_1 \cos\theta + x_2 \sin\theta$, and the variance of a sum, from probability, gives

$$\mathrm{Var}(z) = \cos^2\theta\,\mathrm{Var}(x_1) + \sin^2\theta\,\mathrm{Var}(x_2) + 2\sin\theta\cos\theta\,\mathrm{Cov}(x_1, x_2) = 1 + r\sin 2\theta$$

using $\mathrm{Var}(x_1) = \mathrm{Var}(x_2) = 1$ and $\mathrm{Cov}(x_1, x_2) = r$ for standardised columns, and $\cos^2\theta + \sin^2\theta = 1$, $2\sin\theta\cos\theta = \sin 2\theta$.

The variance is largest when $\sin 2\theta = 1$, at $\theta = 45°$, the diagonal along the band, where it is $1 + r = 1.859$. It is smallest at $\theta = 135°$, across the band, where it is $1 - r = 0.141$. On the spend axis itself, $\theta = 0$, it is 1, as it must be for a standardised column.



Left and middle: 60 accounts and their shadows on two lines, the spend axis and the diagonal. Look at how much wider the shadows spread on the diagonal. Right: the variance of the shadows for every direction from 0° to 180°; its peak at 45° is the first component, its lowest point at 135° the second.

The total variance of the two columns is $1 + 1 = 2$, and the two components split it as $1.859 + 0.141$. So the first component alone carries $1.859/2 = 92.97\%$ of the variance: one number per account, its shadow on the diagonal, describes both columns almost completely.

4.3 Any number of columns: the eigenvectors of the covariance matrix

With eight columns the search is over directions in eight dimensions, and there is no angle to sweep. Linear algebra has a ready-made answer, which we state rather than derive:

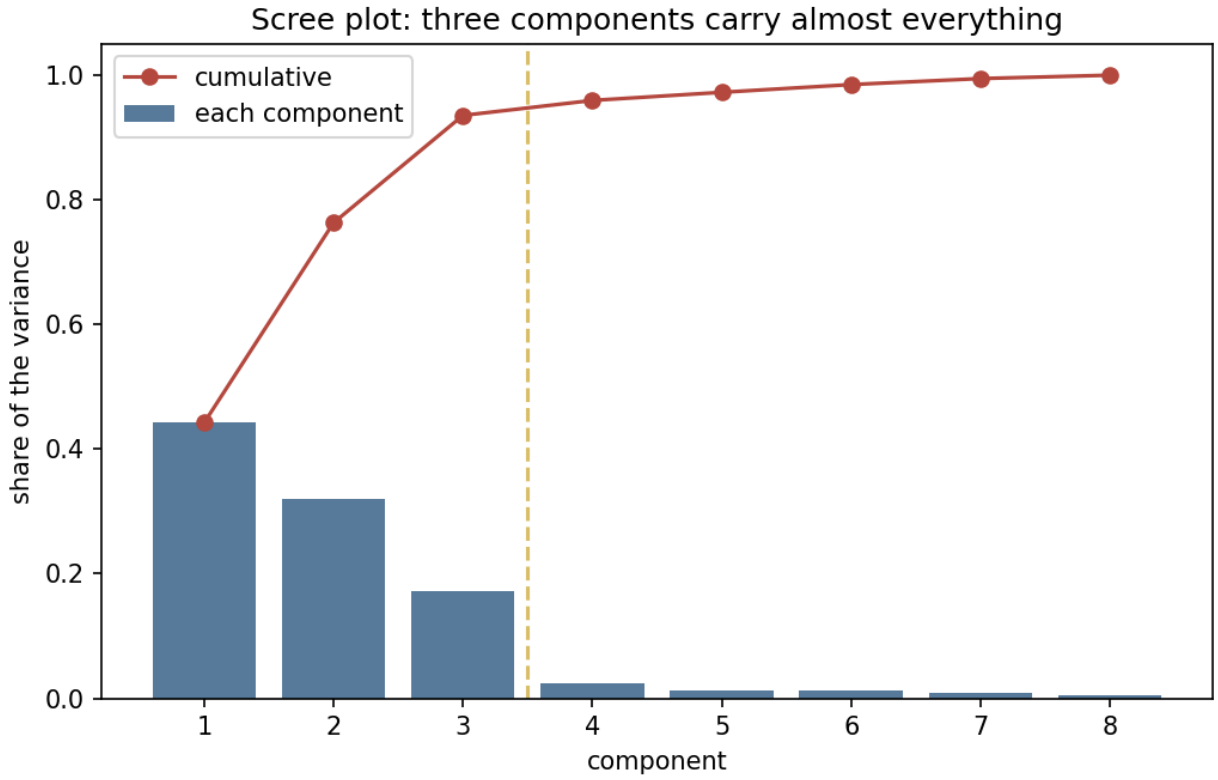
> **The principal components are the eigenvectors of the covariance matrix of the data, and the eigenvalue of each one is the variance of the shadows along it.** Sorted by eigenvalue, largest first, they are the first, second, third component, each at right angles to the ones before.

Check it on the two columns. Their covariance matrix is $\begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$, whose eigenvalues are $1 + r$ and $1 - r$, with eigenvectors $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$: the two diagonals, exactly what the sweep found. Notebook 03 confirms it a third way, with scikit-learn's `PCA`, whose `explained_variance_ratio_` reports each component's share of the total variance: 0.9297 and 0.0703.

The advanced lesson derives the statement from the variance-maximisation problem, and computes PCA a second way, from the singular value decomposition of the data.

4.4 How many components

PCA on all eight standardised columns gives eight components. The **scree plot** draws each one's share of the variance; the components worth keeping come before the shares drop to almost nothing.



Bars: each component's share of the variance. Line: the share of the first 1, 2, 3 … together. Look at the drop after the third bar, marked by the dashed line: three components carry 93.5% of the variance, the fourth adds only 2.4%.

The scree plot has recovered, from the data alone, the number `retail_data.py` built in: three tendencies drive the eight columns. Real data is rarely this clean, and the drop is this sharp only because the generator made it so, but the reading is the same: keep the components before the drop.

The predictable mistake: to keep components until some round share is reached, 95% or 99%, regardless of the plot. The instinct is reasonable, since keeping more variance loses less information. But past the drop, the extra components carry each column's own noise, not structure, and section 5 needs exactly that separation between structure and noise.

---

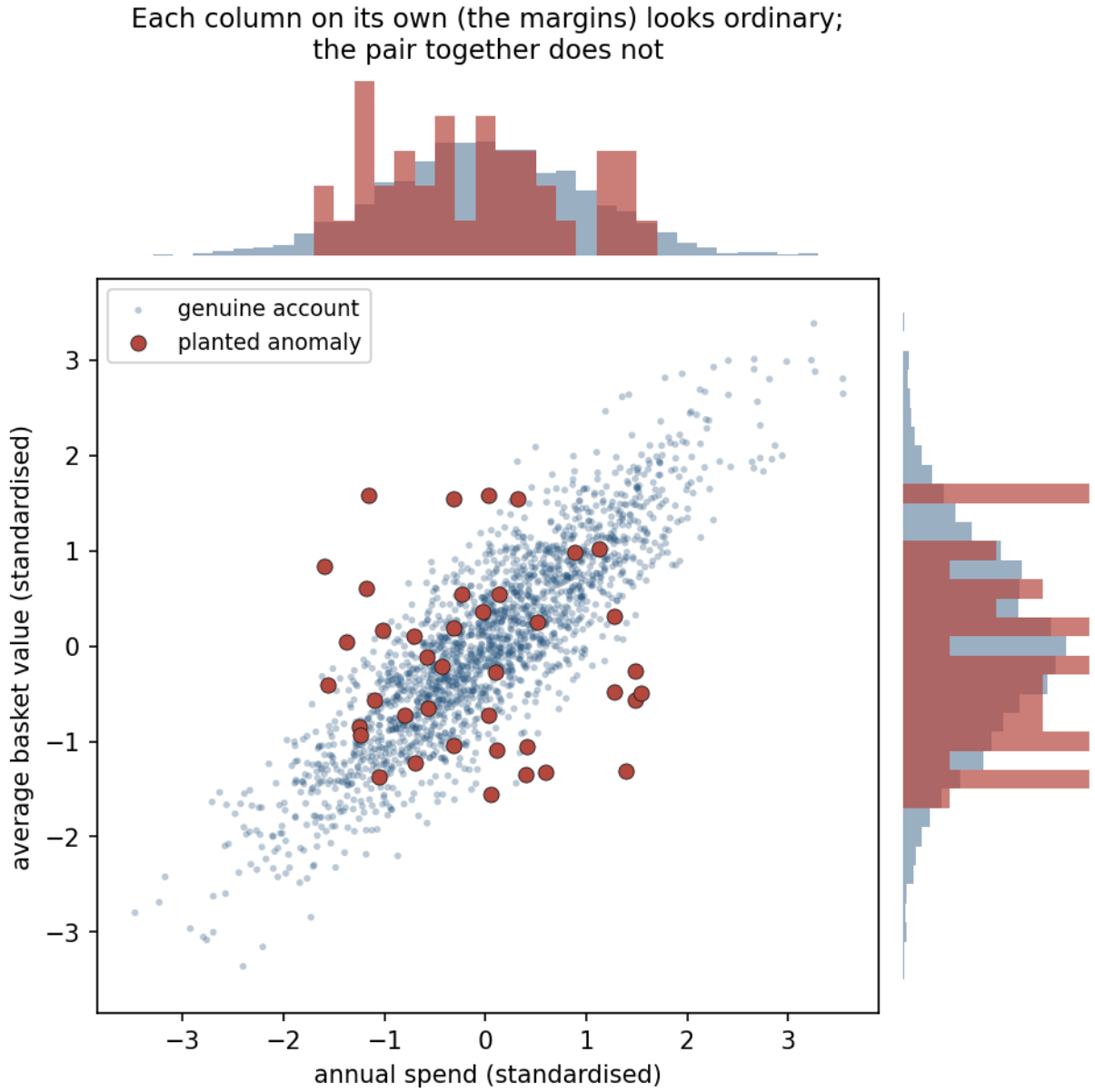
5. Anomalies that break the pattern

5.1 Every value ordinary

Forty of the 2,000 accounts (2%) were taken over by fraudsters. For each of the eight columns, `retail_data.py` drew the fraudulent value at random from inside the range genuine accounts occupy, between their 5th and 95th percentiles, and independently of the other columns. Notebook 03 checks it: every value of every anomaly lies inside that range. No single column gives them away, so no rule of the form "flag a spend above X" can find them.

What the fraudster could not copy is how the columns move together. A genuine account with a

high spend also has a large basket, because one tendency drives both; a fraudulent account's spend and basket were drawn separately.



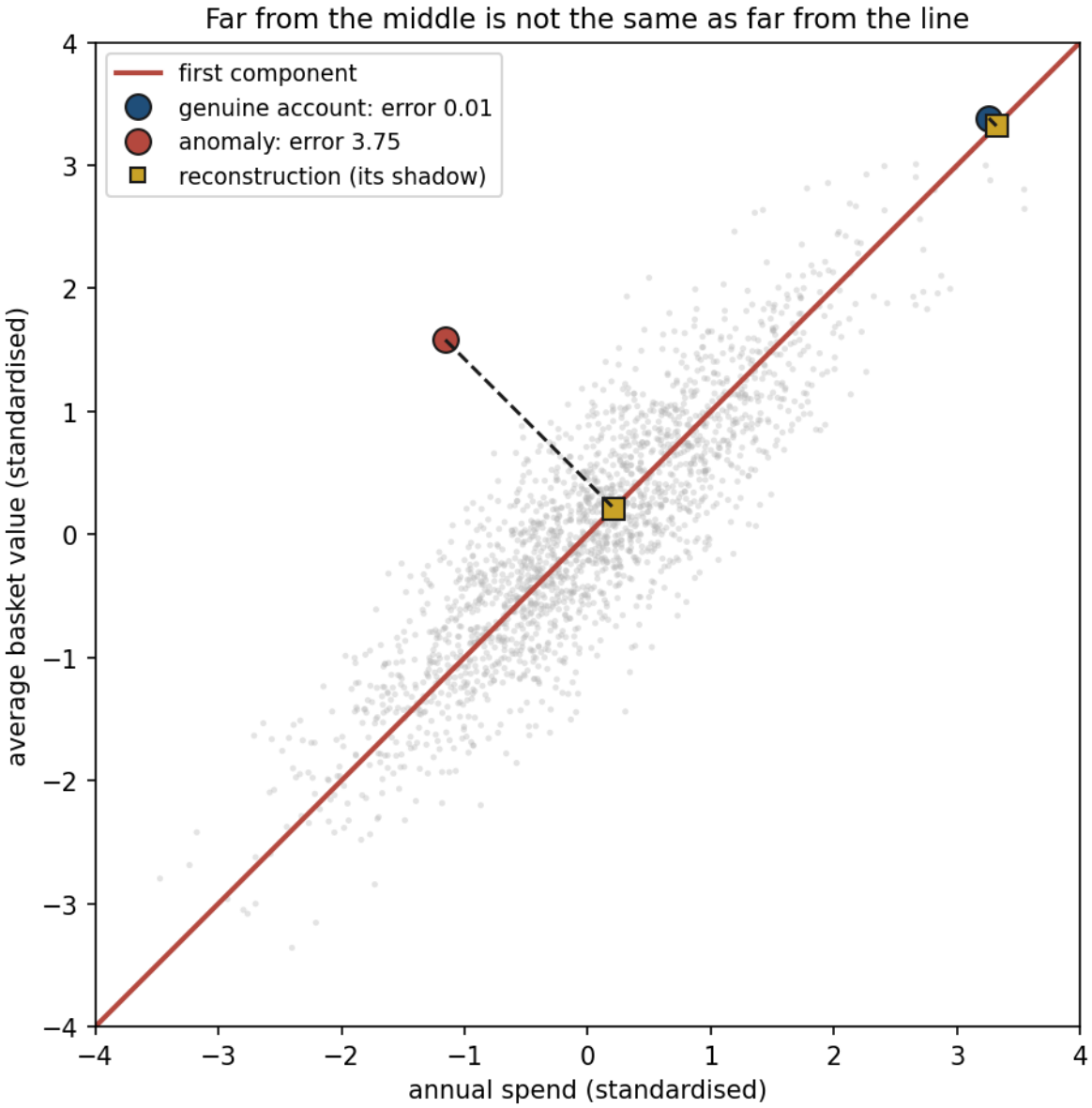
The two columns of section 4.2, with the 40 anomalies in red. Look first at the margins: each column's anomalies (red bars) sit inside the genuine range (blue bars). Then look at the scatter: many anomalies fall off the band that every genuine account follows.

5.2 Reconstruction error

On two columns, keep only the first component, and ask each account to be rebuilt from its shadow alone. The **reconstruction** $\hat{x}$ of an account x is the point on the line where its shadow fell. For an account that follows the band, the reconstruction is close to the real point; for an account off the band, it is far. The squared distance between the two is the account's **reconstruction error**:

$$e = \|x - \hat{x}\|^2$$

In scikit-learn, `transform` computes the shadows and `inverse_transform` places them back in the original columns, so the reconstruction is `pca.inverse_transform(pca.transform(X))`.

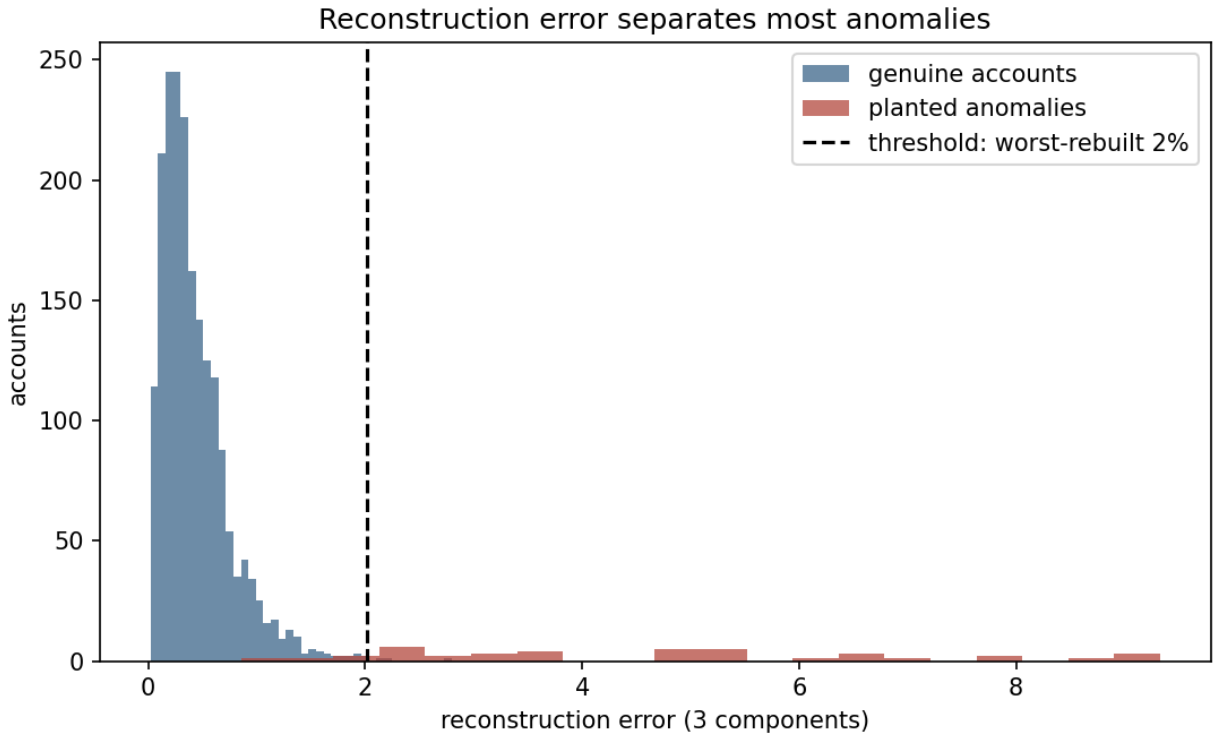


Two accounts and their reconstructions (gold squares). Look at the distances: the genuine account is the farther of the two from the middle, 4.70 against 1.96, yet its error is 0.01, because it lies on the line; the anomaly's error is 3.75, because the line cannot describe it.

That picture is the whole detector, and its point is in its title: **far from the middle is not the same as far from the pattern.** In eight columns, the line becomes the space spanned by the three components kept in section 4.4, and an account is suspicious when those three components cannot rebuild it.

5.3 37 of 40

Notebook 03 fits PCA with three components to the eight standardised columns, reconstructs every account, and flags the 2% with the largest reconstruction error, 40 accounts, since Aurora's fraud team estimates that about 2% of accounts are compromised.



Reconstruction error of every account, genuine in blue and anomalies in red, with the threshold dashed. Look at how few red bars lie left of the line: most anomalies rebuild far worse than any genuine account. The mean error is 4.676 for anomalies against 0.431 for genuine accounts.

37 of the 40 anomalies are among the 40 flagged accounts. So 37 of the 40 flags are right, and 3 anomalies are missed.

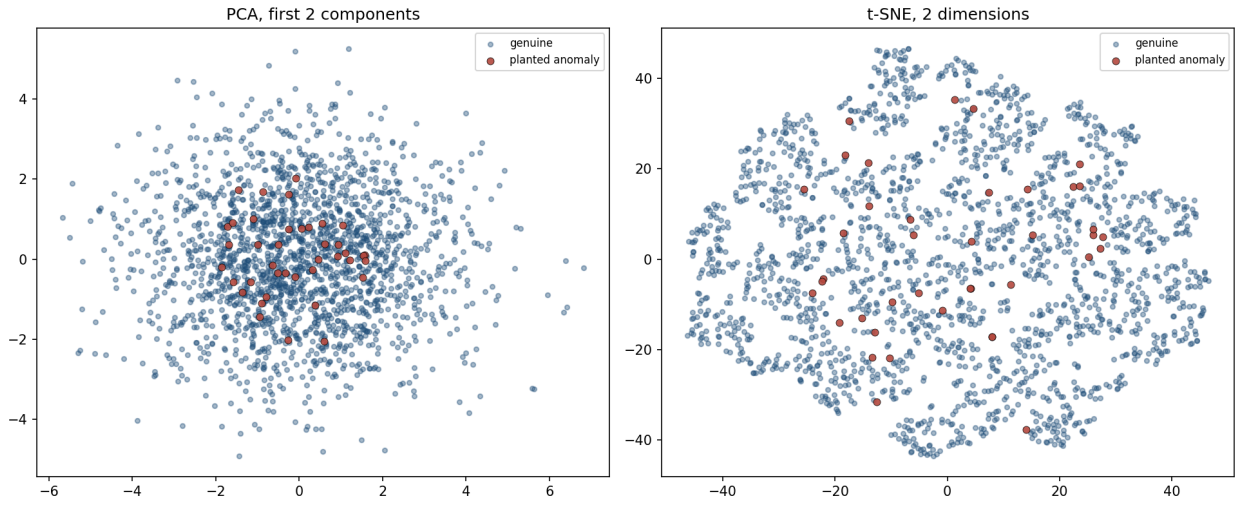
37 is one batch of accounts. Notebook 03 regenerates the table at seven other seeds, the same process with different individual accounts, and repeats the identical procedure. The count caught runs from **32 to 38** of 40, with a median of 36. Carry 37 as the memorable figure and the range as what it means: the detector finds around nine anomalies in ten, and which nine depends on where this quarter's fraudsters happened to land. Quoting "37 of 40" as a property of the method would be quoting one quarter's luck, the lesson 5 point about a single split, here without any y.

5.4 Why a 2-D picture misses them

The natural move is to squash the eight columns into two and look for the anomalies at the edge of the picture. Notebook 03 tries it two ways.

The first is PCA itself, keeping two components. The second is **t-SNE** (t-distributed stochastic neighbour embedding), a method built for pictures: it places the points in two dimensions so that points which were close neighbours in the original columns stay close. It does not try to keep large

distances, so the gaps between distant groups and the overall layout of a t-SNE picture mean little; and it has no `transform` for new data, so placing one new account means running the whole method again. It is a tool for looking, not for measuring.



The 2,000 accounts on PCA's first two components (left) and on t-SNE's two dimensions (right), anomalies in red. Look for the red points at the edges: there are hardly any. They sit among the genuine accounts, and if anything nearer the middle.

On the PCA picture the anomalies sit, on average, at 1.30 from the middle, against 2.20 for genuine accounts: **closer**, not farther. t-SNE shows the same. Flagging the points at the edge would catch the wrong accounts.

The reason is the one section 5.2 drew. A fraudulent account's values were drawn independently and never extreme, so on the directions along which genuine accounts spread out together, its values partly cancel, and its shadow lands near the middle. What gives it away is not its distance from the middle but how badly the kept components rebuild it, and a 2-D picture has already thrown that information away.

The predictable mistake, and the one the exercise builds on: to treat "plot it and look for the points at the edge" as the anomaly detector. The instinct is not stupid: on one column, an anomaly usually *is* at the edge, and notebook 01 showed a scatter plot finding groups faster than any algorithm. But in many columns an anomaly can be far from the pattern while sitting near the middle of every picture.

---

6. Choosing among today's methods

Task	Reach for
Group records with no labels, groups plausibly round	k-means, with several starts
Group records whose groups may be nested, elongated or ring-shaped	DBSCAN, with `eps` from the k-distance plot

Task	Reach for
See the groups at every scale at once, on a few hundred records	hierarchical clustering and its dendrogram
Reduce many correlated columns to a few	PCA, with the number of components from the scree plot
Flag records that break the relationship between columns	PCA reconstruction error
Draw a 2-D picture for a person to look at	t-SNE, or PCA when a fixed, reusable transform is needed

---

Summary

- **k-means** repeats two moves, assign each point to its nearest centre and move each centre to the mean of its points. Its score, the within-cluster sum of squares, can only fall, so it always stops, but not always at the best grouping: one random start in 30 stopped at 1,390.4 against the best 374.1. Run it several times and keep the best.
- k is your choice. The elbow of the WCSS curve and the maximum of the mean silhouette both point to 4 on Aurora's customers, and k-means recovers the true segments with an adjusted Rand index of 0.985.
- On a dense core inside a ring, k-means (ARI -0.046) and Ward hierarchical clustering (ARI -0.062) both fail: both look for compact groups, and can only produce compact halves. **DBSCAN**, which groups points by how crowded their neighbourhood is, recovers the split with ARI 0.941 and leaves 13 ambiguous sessions as noise.
- The silhouette, the only score available without labels, preferred k-means' wrong split (0.401) to DBSCAN's right one (-0.106). It measures compactness, whatever the shape of the data.
- **PCA** finds the directions along which the data spreads most. On two standardised columns with correlation r they are the diagonals, with variances $1 + r$ and $1 - r$; in general they are the eigenvectors of the covariance matrix. Three components carry 93.5% of the variance of Aurora's eight columns.
- **37 of the 40 disguised accounts were caught by reconstruction error** (32 to 38 across eight batches), although every one of their values was ordinary. On a 2-D picture, PCA's or t-SNE's, the same accounts sit closer to the middle than genuine ones: far from the pattern is not the same as far from the middle.

Homework

`Exercises/08_unsupervised_learning.md`, discussed at the start of lesson 9, **Friday 4 December 2026**.

Notation used in this lesson

Symbol	Meaning
k	number of groups (clusters)
μ_j, $c(i)$	the centre of group j; the group of point i

Symbol	Meaning
J	within-cluster sum of squares (WCSS), `inertia_` in scikit-learn
a, b, s	a point's mean distance to its own group, to the nearest other group, and its silhouette
`eps`, `min_samples`	DBSCAN's disc radius, and the number of points (itself included) a core point needs in it
r	correlation between two standardised columns
u, θ, z	a direction of length 1, its angle, and a point's shadow on it
$\hat{x}$, e	a point's reconstruction from the kept components, and its reconstruction error

---

Further reading

Resource	Type	Why read it
scikit-learn: clustering	Official docs	Every clustering method in this lesson, and several more, with the parameters that matter
scikit-learn: decomposition (PCA)	Official docs	PCA's parameters and its variants
Ester, Kriegel, Sander & Xu, *A Density-Based Algorithm for Discovering Clusters* (1996)	Paper	The original DBSCAN paper: section 3.4's algorithm from its source
van der Maaten & Hinton, *Visualizing Data using t-SNE* (2008)	Paper	How t-SNE works, which this lesson leaves out
James, Witten, Hastie & Tibshirani, *An Introduction to Statistical Learning*, ch. 12	Book	Clustering and PCA at the level of this lesson, with more examples
`Advanced/08_unsupervised_learning_advanced/`	This repository	The proofs this lesson states: why k-means stops, PCA derived and computed from the singular value decomposition, four linkages compared, whether `eps` transfers to next week's data